HHAI 2025
D. Pedreschi et al. (Eds.)

doi:10.3233/FAIA250625

Human-AI Collaboration via Trust Factors: A Collaborative Game Use Case

Andrea FANTI [a,1], Francesco FRATTOLILLO [a], Rosapia LAUDATI [a], Fabio PATRIZI [a] and Luca IOCCHI [a]
[a] *Sapienza University of Rome*
ORCiD ID: Andrea Fanti https://orcid.org/0009-0003-0764-3965, Francesco Frattolillo https://orcid.org/0000-0002-2040-3355, Rosapia Laudati https://orcid.org/0009-0000-9508-5235, Fabio Patrizi https://orcid.org/0000-0002-9116-251X, Luca Iocchi https://orcid.org/0000-0001-9057-8946

Abstract. Trust is a well-established and extensively studied concept in the humanitarian field, yet its highly subjective nature, dependence on a wide range of influencing factors, and the absence of a single, unified definition pose significant challenges when attempting to apply it to Human-AI teams. However, being able to assess and recognize when to trust an agent is crucial, especially when deciding whether to delegate tasks or to collaborate on more intricate tasks within a team. Existing work has studied the integration of specific computable factors, which are known to influence trust. In this work, we study how these methods transfer to a more concrete Human-AI collaborative puzzle game. In this setting, the AI agent employs a trust-aware Reinforcement Learning algorithm, while the Human agent's behavior is emulated through the use of a Large Language Model. Our study considers the realistic condition of heterogeneous agents, each skilled in different areas, and we demonstrate that trust-awareness, particularly the ability to make justified decisions about task delegation, is essential for the successful operation of the team. This scenario serves as a testbed for evaluating methods of trust integration and their impact on Human-AI collaboration.

Keywords. Human-Robot Interaction, Trust, Reinforcement Learning, Large Language Models, Planning

1. Introduction

The pervasive use of AI in everyday applications is raising attention to trustworthiness and many works are focusing on studying the role of trust in Human-AI and Human-Robot interaction and collaboration [1,2,3]. The importance of developing trustworthy AI systems [4,5] is crucial and, to this end, it is desirable to design and develop trust-aware AI systems, i.e., AI systems with decision making capabilities based on the level of trust that users interacting with them have in the system. Moreover, trust-aware AI systems enable adaptable and adjustable autonomy that may be extremely important in several application fields.

[1]Corresponding Author: Andrea Fanti, `fanti@diag.uniroma1.it`.

The development of trust-aware AI systems can be significantly supported by the use of computational models of trust in Human-AI interaction. More specifically, a computational model of trust allows for the definition and implementation of trust-aware algorithms able to understand and predict the engagement of humans with AI systems and, in particular, to study the human factors related to trust that contribute to the acceptability of AI-based systems [6]. However, the general concept of trust is difficult to represent in a computational model, since the term *trust* relates to many different meanings that cannot be easily encoded and measured [7].

In this work, we consider *trust factors* as measurable factors that are related with and influence trust in agent-agent interaction. Trust factors can be effectively represented and inserted in a computational model aiming to generate agent-agent interactive behaviors that depend on such trust factors.

More specifically, in this paper we aim to demonstrate the use of a *competence trust factor* [8], a computational model to address a collaborative game between a human-like agent and a robot agent. We use Reinforcement Learning [9] as the solution technique, exploiting its ability to find optimal solutions without requiring the knowledge of a precise model of the problem. In this way, we show that by using a trust factor in the agents' model, it is possible to learn optimal strategies based on trust. To demonstrate the effectiveness of the proposed approach, we consider a collaborative game between a human and a robot sharing a common goal that requires specific competencies to be solved. We consider situations in which the human and the robot have different capabilities, so that the optimal collaborative behavior requires one agent to handover sub-tasks to the other agent. We show the capability of our approach to learn collaborative behaviors in which agents are able to estimate the other agent's trust (through measurable trust factors) and take decisions based on such trust factors. With the help of a large language model (LLM) [10], we generate different human-like profiles that are used to train our trust-aware reinforcement learning agent in order to evaluate its performance. Finally, we also employ a classical planning approach to the problem, with the aim of producing an upper bound for the performance of a semantically-varied team of agents in the best-case scenario. In future work, we aim to fully implement the system on a real robotic platform and to validate the proposed approach through user studies.

2. Related Work

This paper focuses on a collaborative Human-Robot Interaction (HRI) where one agent, simulating a human-like behavior, is generated by a large language model (LLM), while another agent, robot-like, is implemented with a reinforcement learning approach. Both agents are *trust-aware*, as their decisions are influenced by the level of trust they have in their partner. This section is dedicated to reviewing the literature on trust applied to HRI, trust in the context of MARL, and the use of LLMs as tools for simulating human decision-makers. We will also highlight gaps in the existing literature and emphasize the contributions of this paper.

LLM for simulating human behavior Due to the complexity related to recruiting and maintaining users in research studies [11], and due to the vast knowledge of the Large Language Models (LLMs), the latter are increasingly being used to simulate human behavior in various user studies [12]. This simulation of complex human behavior has al-

ready been studied in different contexts such as group actions [13], cognitive reasoning [14] and decision-making tasks [15] but usually they requires constant interactions with the LLM in order to generate new actions. This represents one of the main challenges [16] since LLMs are highly resource-intensive, making them suboptimal for processes that require many iterations, such as in reinforcement learning tasks [17]. There are works that tackle this problem by generating code snippets that simulate required behavior and that can be directly executed such as agents for playing starcraft [18] or for robotic control [19,20]. LLM has also been used in reinforcement learning as information processor, reward designer, decision-maker, and generator [21], but to the best of our knowledge, no existing work uses LLMs to generate code for rule-based agents and use them to train reinforcement learning agents.

Trust in Human-Robot Interaction Trust has been extensively studied in different discipline such as philosophy [22,23] and human-computer interaction (HCI) [24,25]. Exploring trust in Human-Robot Interaction (HRI) is essential for ensuring successful collaboration in various environments. Getting the right balance of trust is critical—too little trust can lead to disuse, where humans avoid using systems that could be helpful, while too much trust can result in over-reliance, where humans place excessive confidence in robots, potentially accepting flawed decisions or suggestions [26]. Given trust's complexity, researchers often analyze it in specific application contexts, breaking it down into distinct components rather than treating it as a singular, unified idea. These components can be linked to the robot, the human, or the surrounding environment [27,28]. Many approaches to measuring trust focus on subjective tools, such as surveys and questionnaires [29,30,31]. While useful for capturing trust parameters in human-robot interactions, these methods don't provide the precise definitions needed for robot-centered trust models. Despite some preliminary works that try to fills this gap [32,33,34,35], automatic recognition of trust remains an open challenge.

Trust in Multi-Agent Reinforcement Learning Despite the large literature on trust in the context of human-human and human-robot teams, trust in the context of human-RL agents is a very underexplored area. Some works use the concept of trust in reinforcement learning, but not as variable that influence the learning process or the decision making of the RL agent. These works include using Reinforcement Learning frameworks such as Markov Decision Processes (MDPs) [36] as a tool to quantify trust [37] or using trust to reach a consensus [38]. On the other hand, there are works that directly use trust to influence the learning and decision making processes in MARL. Following the general structure of trust formula in the context of Human-AI RL framework [39], Di Rienzo et al. (2024) [40] applied this concept to a specific trust factor influenced by the ability to predict other agent's intentions. On a similar concept, Frey et al. (2024) [41] proposed a definition of trust that involves agents choosing to act as another would in the same situation. Finally, Contino et al. (2024) [8] introduced the "Competence Trust Factor" which is a function that depends on the ability of an agent to complete a specific task. Despite that, the previous work uses these concepts in the context of homogeneous teams, where every agent is a RL agent. In this paper, we consider a more realistic Human-AI/Robot scenario, where agents are highly heterogeneous and humans are simulated during the training phase by an LLM instead of being another RL agent. This opens the path to new opportunities to apply trust-aware RL to Human-AI/Robot teams.

Contributions The main contributions of this paper are the following:

- We propose an innovative application of trust-aware Reinforcement Learning in a Human-AI/Robot Interaction scenario. The RL algorithm learns, during its interactions with its partner, when to trust and when not to. Through ablation studies, we have demonstrated that the inclusion of the trust factor within the RL algorithm enhances performance, both in terms of task success and, more importantly, by improving completion times.
- We have developed a collaborative puzzle game that requires knowledge in various topics to be solved, in order to validate our solution. This game serves as an ideal testbed, as it necessitates collaboration only when different players possess distinct relevant knowledge and trust their partners' abilities when needed.
- We have used a large language model to automatically generate code scripts that simulate the behavior of different human profiles, and used these scripts during training of our RL agent. Human profiles are generated by considering various factors, such as their knowledge of a specific topic and their tendency to trust a game partner.

3. Background

In this section, we provide some formal definitions useful to understand the proposed approach.

3.1. Multi-Agent Reinforcement Learning

Multi-Agent Reinforcement Learning problems are usually described as Markov Games (MGs), which are generalizations of Markov Decision Processes to the multi-agent setting [42] A Markov Game (MG) is defined as a tuple $\langle N, S, A, T, R, \gamma \rangle$, where N is the number of players (agents), S is the set of environment states, shared by all agents, A is the set of joint actions $A = A_1 \times \cdots \times A_n$, where A_i is the set of actions of agent i, $T : S \times A \times S \to [0,1]$ is the joint transition function returning for $T(s,a,s')$ the probability of the transition from state s to s' under the joint action a, $R : S \times A \times S \to [0,1]$ is the common reward function of all agents, and $0 < \gamma < 1$ is the discount factor. A solution for the MARL problem is to find N functions called policy $\pi(s)$, which is a function that returns a distribution over action given the current state, that maximize the future cumulative reward $G = \sum_{k=t}^{T} \gamma^k R(s_k, a_k)$.

3.2. Classical Planning

Classical Planning [43] is a logic-based approach that deals with the problem of finding an action sequence able to take a dynamic domain from an initial to a desired goal state. In its basic variant, actions are *deterministic*, i.e., the outcome of their execution in a state is known before the actual execution. An example is to find a sequence of moves to solve Rubik's cube starting from a given initial configuration.

A *planning domain* $\mathscr{D}$ models the environment of interest as a pair $\langle F, A \rangle$, where:

- F is a finite set of *fluents*, i.e., propositions that can be true or false in each state (e.g., a given cell on some face of the cube is green);

- A is a finite set of *actions* available in the environment (e.g., rotating a specific slice of the cube in a given direction), modeled as tuples $a = \langle pre, eff \rangle$, with *pre* the *precondition* of a, i.e., a logical specification of which fluents need to be true and which ones false in order for a to be executable, and eff the *effect*, i.e., a logical specification indicating which fluents become false and which ones true after executing a.

A *state* $s \subseteq F$ represents the state of the environment at a given time point and contains all and only the fluents that are true at that point. A *planning problem* is defined as a tuple $\mathscr{P} = \langle \mathscr{D}, s_0, G \rangle$, where $\mathscr{D}$ is the planning domain of interest and:

- $s_0 \subseteq 2^F$ is the *initial state*, containing all and only the fluents that are initially true (e.g., color of each cell of the cube);
- G is the the problem's *goal*, i.e., a logical specification expressing the desired properties to reach (e.g., all cells on every face must have same color).

Solving a classical planning problem $\mathscr{P}$ amounts to finding a sequence of actions $\pi = a_1 \cdots a_n$, called a *plan*, s.t. a_i is executable in state s_{i-1} and leads the domain to state s_i, and s_n satisfies the goal condition G.

3.3. Trust Factors and Trust-aware MARL

In order to create a trust-aware reinforcement learning agent, a notion of trust or trust factor should condition the learning or the decision making of an agent. A Trust factor is usually a function with the following structure [39]

$$TrustFactor(y, \Gamma | x, K) \in [0, 1] \tag{1}$$

where x is called a *trustor* and is the agent currently evaluating the trust, y is the called the *trustee* and it's the agent being trusted, Γ is the current task and K is a set of knowledge available. Contino et al. (2024) [8] introduced the competence trust factor in the context of multi-agent reinforcement learning to estimate the trust between RL agents by considering their ability to correctly complete a task when their intention is to complete that task. This trust factor is independent from the trustor, hence it has been formulated as:

$$TrustFactor(y, \Gamma | K) \in [0, 1] \tag{2}$$

4. Collaborative Semantic Memory Game

In the context of this work, we formulate a collaborative game based on the memory game, in which the players have a shared common goal and the solution requires a proper integration of the different skills of the agents. In the standard memory card game (also known as "Pexeso" or "Concentration") [44], a deck of $2n$ cards depicting n images, such that each image appears twice, is shuffled and laid out face down on a table. In the solitaire version, the goal of the game is to sequentially uncover all pairs of cards with the same symbol in as few moves as possible. With multiple players, instead, the goal is usually to uncover more pairs than each opponent. Each move consists in choosing

two face down cards and turning them face up. If the two cards match, they are collected by the active player; otherwise, they are turned face down again, so that players cannot see them and must rely on their memory [45]. The original multi-player version can be formalized as a zero-sum, sequential finite stochastic game with perfect information [44], requiring a good memory for optimal play.

The version we propose here, instead, differs from the above version in two major aspects: (1) it is collaborative, featuring a single two-player team; (2) cards are not paired by matching symbols but through a semantic association, such as a country with its capital (e.g. Italy $\leftrightarrow$ Rome) or a mathematical expression with the value it evaluates to (e.g. $3+3 \leftrightarrow 6$). The former is implemented by having the two players take turns in choosing the next card to be turned face up, with the goal of collaboratively collecting all pairs of cards in as few moves as possible. The latter means that a good player should not only remember the position of each card, but also needs to have the required semantic knowledge to correctly associate the cards. Most notably, this means that two players that have complementary semantic knowledge must collaborate in order to play the game optimally. Finally, note that if an agent sees both cards that make up a match (such as "Rome" and "Italy"), but it does not have the necessary semantic knowledge to recognize that they match, from its point of view it is as if both cards were still unknown.

4.1. Human-like LLM Agent

The human-like agent is an agent simulating human behaviors using a large language model (LLM), through precisely instructed prompts. The use of LLM to define human-like agents is motivated by the need of generating experience to train the RL agent, without requiring humans to get involved in such a long and tedious activity.

The goal is to simulate the profile of a player with specific characteristics. A player can have a lack of confidence in its own ability and decides to mainly handover the choice to its partner, or it can be very confident and always pick the card by itself. Additionally, the agent can also be instructed to have specific knowledge such as being good in math but bad in geography or vice versa. The human-like agent is instructed beforehand about the level of its partner in specific categories. To this end, we initially considered prompting the LLM agent at each iteration, but this method has some major drawbacks. Reinforcement Learning agents require thousands of iterations in order to learn a task due to their ability to learn through trial and error. Most commercial LLMs that can be accessed through API have a limit on the number of prompts that can be used; additionally, extensive use can have a bad environmental impact [46]. On the other hand, most of the open source LLM [47,48,49] available have a huge number of parameters, making them prohibitive to use on common computers or laptops.

Motivated by these problems and by the nature of the proposed game, we decided to use a different approach. We asked an LLM to directly implement a function in Python that models human-like behaviors according to some features. The function should receive in input the current state of the environment and should return an action. Then this function is used during training without the need of accessing the LLM. More specifically, we generated a set of such functions beforehand and use them during training to guarantee variability of human behaviors when training the RL agent. Both the template that we used to generate the human-like agent and an example of the generated behavior are shown in Figure 1.

(a) ChatGPT Prompt

"Suppose you are a human that wants to play a game of memory, where the association have a semantic meaning instead of being the same symbol (for example one card could be France and its associated card is Paris, its capital or one card could be 2+2 and its associated card is 4). You are playing this game with your partner, which is another agent ...[description of the rules of the game] Suppose that you are roleplaying as a human and you can have 3 different level of knowledge about capitals-nation associations and math operations associations, "bad" you don't know nothing, "medium" you know something, "good" you are very good. Similarly suppose you also have three different level of trust on your partner "low trust", "medium trust" or "high trust".

At each timestep you receive a python dictionary with the current state of the board. The initial state is: {0:"unknown", 1:"unknown", 2:"unknown", 3:"unknown",4:"unknown",5:"unknown",6:"unknown",7:"unknown", 8: "None"} For the first 7 elements of the dictionary... [description of the state] At each time step, you can pick a number between 0 and 7 ... [description of actions and their consequences].

I want you to write a script that has some elements of stochasticity ... [description of conditions affecting the stochasticity]. The behaviour you generate must handle at least the following possible cases and their combinations ... [description of possible combinations]"

(b) Generated Behavior

```python
def humanDecision(state):
known_cards = {k: v for k, v in state.items() if v not in ["unknown", "solved", "None"] and k != 8}
uncovered_card = state[8]

capitals_nations = {"Paris": "France", "France": "Paris", "Rome": "Italy", "Italy": "Rome"}
math_operations = {"2*(5-3)": "4", "4": "2*(5-3)", "7*8": "56", "56": "7*8"}

# If no known cards and no uncovered card, either uncover a random one or handover
if not known_cards and uncovered_card is None:
    return random.choices([8, 9], weights=[0.3, 0.7])[0]  # Prefer handing over due to high trust

# If an uncovered card exists
if uncovered_card:
    if uncovered_card in capitals_nations:  # Good at capitals-nations
        associated_card = capitals_nations[uncovered_card]
        if associated_card in known_cards.values():
            return next(k for k, v in known_cards.items() if v == associated_card)  # Match it
        else:
            return random.choices([8, 9], weights=[0.7, 0.3])[0]  # Try uncovering a card first
    elif uncovered_card in math_operations:  # Bad at math
        return random.choices([8, 9], weights=[0.2, 0.8])[0]  # Mostly handover due to low knowledge

# If there are known cards (but no uncovered card)
for card, index in known_cards.items():
    if card in capitals_nations:  # Good at capitals-nations
        return index  # Try uncovering the known good association first

# If we have known cards but only bad at them
return random.choices([8, 9], weights=[0.1, 0.9])[0]  # Prefer to handover due to low trust in ability
```

Figure 1. (a) A template of the prompt used to generate the human-like agents. The 3 dots indicates that the sentence will continue and the text between the square brackets represents a synthesis of the content of the original prompt, for brevity. (b) An example of the output scripts generated by ChatGPT for the human like agents. This script implements the behavior of H_{good}, a human-like agent that is good at matching capital cities and countries, and has high trust in their partner.

4.2. Robot-like Reinforcement Learning Agent

The robot agent is trained using tabular Q-learning [50], a classic reinforcement learning algorithm with some guarantees of optimality [51]. The choice of a single-agent RL algorithm is well-motivated by the fact that all human-like policies used as teammates in this work are stationary, reducing the MARL problem to a single-agent one. The state space of the agent is augmented with a discretized representation of the trust factor described in Section 4.3. Consequently, the training procedure and the output policy are based on such a trust factor, thus implementing a *trust-aware Q-learning* approach. Albeit the appropriate level of discretization is domain-dependent and may, in general, affect the final performance, our formulation and experimental setup are agnostic to this choice as this only affects the size of the state space of the RL agent. The integration of this trust factor is also motivated by the possibility of performing quantitative and qualitative analisys on the learned behavior from the high-level point of view given by the trust factor dy-

namics, which is not possible with a non-trust-aware RL agent. Finally, while expecting that an agent integrating additional information will obtain better overall performance is sensible, this is not true in general. In fact, any information with turns out to be useless or deceptive to an RL agent may often lead to reduced final performance and/or sample efficiency.

4.3. Competence Trust Factor in the Collaborative Game

In this work, we adapt the definition of trust factor in equation 2 by also considering that the proposed semantic memory game has Markovian properties, i.e. its future evolution only depends on the current state and not on the past ones. This is a consequence of the fact that the robotic agent has perfect memory and remembers all the cards that have been previously uncovered. By using this assumption, the trust factor can be written as:

$$TrustFactor(y,(c,match)|K) = \mathbb{P}(match|c) \tag{3}$$

where c represents the category of a card. The trust factor is given by the trustee's ability to correctly complete a match given that the card has a specific category, which means that it is proportional to the probability of having a match given the category.

5. Planning Domain

We tested the effectiveness of the competence trust factor also in the setting of planning, used as an approach to encode (and solve) the Semantic Memory Game. To set a reference for the RL approach, we consider the best-case scenario, where each player can see the value of every card (even though laying face-down). However, a player cannot match two cards unless (s)he has high trust on the corresponding topic.

The state of the game is modeled through suitable fluents. For every card, one fluent (`faceup`) models whether the card is laying face-up or face-down, one models the value of the card (`has_value card value`), another the matching card (`semantically_linked card1 card2`), and one the relevant category (`category`). Most importantly, we also model the players' trust level. Although in principle there could be many, for simplicity we use only two level, and respective fluents, `trust_high` and `trust_low`. Other bookkeeping fluents are present (such as `active`, modeling turns), but are omitted for brevity.

The available actions include: handing the turn over to the other player (`hand_over_control`), flipping the first card (`flip_first_card`), flipping the second card (`flip_second_card`). While the first and second actions are not affected by the player's trust level, the last one is. Specifically, when the first card is face-up, a player can flip the matching card only if (s)he has high trust over the card's topics, otherwise (s)he must hand the turn over (or flip the wrong card). We report below the specification in PDDL (Planning Domain Definition Language) of the handover action:

```
(:action handover_control
 :parameters (?current - agent ?next - agent ?cat - category)
 :precondition (and (active ?current) (trust_high ?next ?cat))
 :effect (and (not (active ?current)) (active ?next)))
```

PDDL is a very intuitive language for the specification of planning domains and problems, widely adopted in the Planning community. Here we omit a formal description of PDDL, referring the interested reader to [52] for an introduction.

With the proposed formalization, a rather simple plan is obtained, namely that where the initial player matches first all the pairs concerning a category that (s)he has high trust on, and then hands control over to the other player, who matches all the remaining pairs, over which (s)he has high trust (we assume that for every category, at least one player has high trust over it). The execution of this plan provides for a best theoretical result that is useful to assess the performance of the RL-based approach.

6. Experiments

For all our experiments, we consider an instance of the Semantic Memory Game described in Section 4 in which the deck is composed of a total of 8 cards. These contain two categories of semantic associations: capitals-and-countries, and math-expressions-and-values. Each category is composed of 2 pairs of matched cards. To give an upper bound to the best-case performance of a semantically-varied team of players, we compute an optimal deterministic plan as described in Section 5.

6.1. Human-like LLM Agents

We use ChatGPT [53] to generate code snippets representing different human-like players. Figure 1 reports both the template of the prompts used to generate the code snippets and an example of the generated behavior scripts. In particular, we instruct ChatGPT to produce two human-like agents: H_{good} that is good at matching capitals-to-countries and bad at math-expressions-and-values, and H_{bad}, that is bad at both. As the use of an LLM to generate human-like behaviors is not the main focus of this work, we do not investigate with different LLMs, which we conjecture would lead to similar results.

6.2. Robot-like Reinforcement Learning Agents

To study the impact of the trust-aware RL approach with respect to a naive RL approach, we train two RL agents: RL_{CTF}, which receives the CTF of its teammate in addition to the board state, and RL, which only observes the board state. For both these agents, we artificially impair their competence in choosing cards of the capitals-and-country category, meaning that their competence will be at best complementary to that of the human-like agents, and not overlapping. Both agents are trained by extracting a random human-like agent at the start of each training episode. In addition, we also train two baseline agents, both of which play the game with no teammate: RL_{omni}, which has no competence impairment, and RL_{solo}, which retains the competence impairment but still plays alone. We reward all agents with the same scheme: a match results in a reward of $+1$, winning the game results in a reward of 100, and any other move (including handing over control to the teammate where applicable) results in a reward of -1.

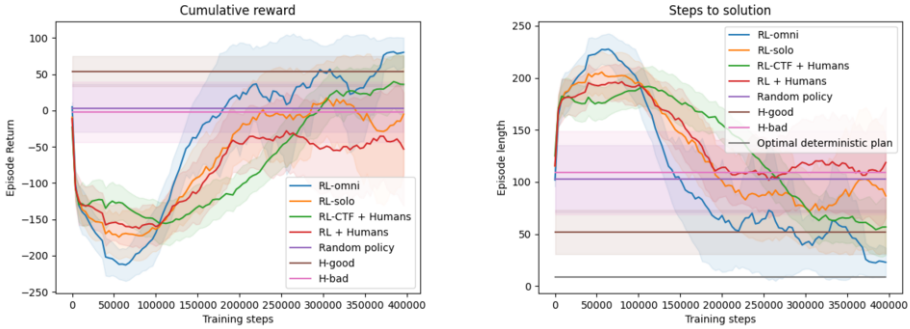

Figure 2. Cumulative reward and steps to solution during training. The trust-aware agent (green) exhibits significantly better performance with respect to its non-trust-aware counterpart. Training curves are averaged over 5 different seeds, and smoothed with a sliding window of 10 samples. Fixed policies are evaluated over 1000 episodes. All curves besides the optimal deterministic plan are surrounded by a 1σ shade.

7. Results and Discussion

Table 1 and Figure 2 report the performance respectively of the best Reinforcement Learning agents seen during training, and of the Reinforcement Learning agents evaluated at fixed intervals during training, compared with the non-learning-based agents and optimal deterministic plan. The results clearly show how the trust-aware RL agent RL_{CTF} obtains significantly better results than: (i) the naive RL agent RL with both human-like teammates H_{good} and H_{bad}; (ii) both human-like agents alone; and (iii) the competence-impaired RL agent RL_{solo}. This means that teaming up with the trust-aware agent is convenient for both human-like agents and for RL_{CTF} itself, demonstrating how a trust-aware learning approach greatly boosts Human-AI collaboration when both parties have only partial competence in a shared task. In particular, the trust-aware RL agent is able

Table 1. Performances of the best agents seen during training when paired with both the good and bad human-like agents, compared with the baseline policies and optimal deterministic plan. The trust-aware RL agents significantly surpass the performance of the non-trust-aware RL agents with both human-like agents. Interestingly, the non-trust-aware agent plays worse when paired with the good human-like agent; we provide more details on this in the discussion. As expected, the team of a trust-aware agent and a human-like agent surpasses the performance of the corresponding human-like agent alone, with the best combination being the trust-aware agent and good human-like agent. Results are reported as mean $\pm$ standard deviation over 1000 test episodes.

Configuration	Cumulative Reward	Steps to solution
Optimal deterministic plan	–	9
Random policy	3 ± 32	104 ± 32
H_{good}	55 ± 21	52 ± 21
H_{bad}	-3 ± 43	110 ± 41
RL_{omni}	93 ± 2	14 ± 2
RL_{solo}	70 ± 41	36 ± 30
$RL_{\text{CTF}} + H_{\text{good}}$	73 ± 60	29 ± 42
$RL_{\text{CTF}} + H_{\text{bad}}$	61 ± 59	42 ± 42
$RL + H_{\text{good}}$	5 ± 32	101 ± 32
$RL + H_{\text{bad}}$	35 ± 105	60 ± 73

to adapt its policy to the competence of its teammate such that the team performance improves considerably.

In contrast, the naive RL agent is not able to properly adapt to different human-like teammates, even performing worse when paired up with a good-performing human-like agent. This is due to the training scheme: since the teammate is chosen at random at the start of each episode, and episodes played with H_{bad} are generally longer than those played with H_{good}, the RL algorithm will experience more steps with the former. Consequently, when presented with a teammate that exhibits a radically different behavior, this will lead to poor performance, regardless of the competence of the new teammate. Since the training regime is the same also for RL_{CTF}, this conversely shows how its trust-awareness allows it to learn to properly play with both H_{good} and H_{bad}, balancing the disparity of training scale.

8. Conclusions and Future Work

In this paper, we have adapted and analyzed, within the context of Human-Robot Interaction (HRI), one of the key factors influencing trust: the Competence Trust Factor (CTF). We considered a scenario involving two agents: a robot-like agent simulated through reinforcement learning, and a human-like agent simulated using a Large Language Model (LLM). We introduced a collaborative memory-inspired puzzle game, which requires specific knowledge of certain categories in order to make correct associations.

We assessed the effectiveness of the CTF through experiments with different human-like agents generated by an LLM, each with distinct characteristics such as varying levels of semantic category knowledge. Specifically, we compared the performance of RL agents with and without CTF against various baselines, as described in the previous section. The results demonstrated the utility of incorporating the CTF in human-robot interaction scenarios.

For future work, we plan to test the trust factors in a real-world human-robot user study with actual human participants. Additionally, we aim to apply the decision-making algorithms to a real robot to evaluate its practical effectiveness and, more in general, to validate and extend the approach of developing computational models using trust factors.

We believe that the definition, the implementation, and the experimental evaluation of computational models of trust are key success factors for the development of trust-aware AI systems.

9. Acknowledgments

The work is supported by the Air Force Office of Scientific Research under award number FA8655-23-1-725 and PNRR MUR project PE0000013-FAIR.

References

[1] Lewis M, Sycara K, Walker P. In: The Role of Trust in Human-Robot Interaction; 2018. p. 135-59.

[2] Ueno T, Sawa Y, Kim Y, Urakami J, Oura H, Seaborn K. Trust in Human-AI Interaction: Scoping Out Models, Measures, and Methods. In: Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems. CHI EA '22. New York, NY, USA: Association for Computing Machinery; 2022. Available from: https://doi.org/10.1145/3491101.3519772.
[3] Afroogh S, Akbari A, Malone E, Kargar M, Alambeigi H. Trust in AI: progress, challenges, and future directions. Humanities and Social Sciences Communications. 2024;11. Available from: https://doi.org/10.1057/s41599-024-04044-8.
[4] Kaur D, Uslu S, Rittichier KJ, Durresi A. Trustworthy Artificial Intelligence: A Review. ACM Computing Surveys. 2023;55.
[5] Li B, Qi P, Liu B, Di S, Liu J, Pei J, et al. Trustworthy AI: From Principles to Practices. ACM Computing Surveys. 2023;55.
[6] Mattioli J, Sohier H, Delaborde A, Pedroza G, Amokrane K, Awadid A, et al. Towards a holistic approach for AI trustworthiness assessment based upon aids for multi-criteria aggregation. In: Pedroza G, Huang X, Chen XC, Theodorou A, editors. SafeAI 2023 - The AAAI's Workshop on Artificial Intelligence Safety. vol. 3381. Washington, D.C., United States: AAAI; 2023. Available from: https://hal.science/hal-04086455.
[7] McKnight D, Chervany N. The Meanings of Trust; 1996.
[8] Contino G, Cipollone R, Frattolillo F, Fanti A, Brandizzi N, Iocchi L. Modeling a Trust Factor in Composite Tasks for Multi-Agent Reinforcement Learning. In: Proceedings of the 12th International Conference on Human-Agent Interaction; 2024. p. 195-203.
[9] Sutton RS. Reinforcement learning: An introduction. A Bradford Book. 2018.
[10] Radford A. Improving language understanding by generative pre-training; 2018.
[11] Far PK. Challenges of Recruitment and Retention of University Students as Research Participants: Lessons Learned from a Pilot Study. Journal of the Australian Library and Information Association. 2018;67(3):278-92. Available from: https://doi.org/10.1080/24750158.2018.1500436.
[12] Guozhen Z, Zihan Y, Nian L, Fudan Y, Qingyue L, Depeng J, et al.. Human Behavior Simulation: Objectives, Methodologies, and Open Problems; 2024. Available from: https://arxiv.org/abs/2412.07788.
[13] Park JS, O'Brien J, Cai CJ, Morris MR, Liang P, Bernstein MS. Generative Agents: Interactive Simulacra of Human Behavior. In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. UIST '23. New York, NY, USA: Association for Computing Machinery; 2023. Available from: https://doi.org/10.1145/3586183.3606763.
[14] Shah D, Osiński Bz, ichter b, Levine S. LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action. In: Liu K, Kulic D, Ichnowski J, editors. Proceedings of The 6th Conference on Robot Learning. vol. 205 of Proceedings of Machine Learning Research. PMLR; 2023. p. 492-504. Available from: https://proceedings.mlr.press/v205/shah23b.html.
[15] Zhu X, Chen Y, Tian H, Tao C, Su W, Yang C, et al. Ghost in the Minecraft: Generally Capable Agents for Open-World Environments via Large Language Models with Text-based Knowledge and Memory. CoRR. 2023;abs/2305.17144. Available from: http://dblp.uni-trier.de/db/journals/corr/corr2305.html#abs-2305-17144.
[16] Gui G, Toubia O. The Challenge of Using LLMs to Simulate Human Behavior: A Causal Inference Perspective. SSRN Electronic Journal. 2023. Available from: http://dx.doi.org/10.2139/ssrn.4650172.
[17] Hu B, Zhao C, Zhang P, Zhou Z, Yang Y, Xu Z, et al.. Enabling Intelligent Interactions between an Agent and an LLM: A Reinforcement Learning Approach; 2024. Available from: https://arxiv.org/abs/2306.03604.
[18] Deng Y, Ma W, Fan Y, Zhang Y, Zhang H, Zhao J. A new approach to solving smac task: Generating decision tree code from large language models. arXiv preprint arXiv:241016024. 2024.
[19] Liang J, Huang W, Xia F, Xu P, Hausman K, Ichter B, et al. Code as Policies: Language Model Programs for Embodied Control. In: 2023 IEEE International Conference on Robotics and Automation (ICRA); 2023. p. 9493-500.
[20] Singh I, Blukis V, Mousavian A, Goyal A, Xu D, Tremblay J, et al. ProgPrompt: Generating Situated Robot Task Plans using Large Language Models. In: 2023 IEEE International Conference on Robotics and Automation (ICRA); 2023. p. 11523-30.
[21] Cao Y, Zhao H, Cheng Y, Shu T, Chen Y, Liu G, et al. Survey on Large Language Model-Enhanced Reinforcement Learning: Concept, Taxonomy, and Methods. IEEE Transactions on Neural Networks

and Learning Systems. 2024:1-21.
[22] Baier A. Trust and antitrust. Ethics. 1986 1;96:231-60. Available from: https://www.jstor.org/stable/2381376.
[23] Pouryousefi S, Tallant J. Empirical and Philosophical Reflections on Trust. Journal of the American Philosophical Association. 2023;9.
[24] Bach TA, Khan A, Hallock H, Beltrão G, Sousa S. A Systematic Literature Review of User Trust in AI-Enabled Systems: An HCI Perspective. International Journal of Human-Computer Interaction. 2024;40.
[25] Hoff KA, Bashir M. Trust in automation: Integrating empirical evidence on factors that influence trust. Human Factors. 2015;57.
[26] Parasuraman R, Riley V. Humans and Automation: Use, Misuse, Disuse, Abuse. Human Factors: The Journal of the Human Factors and Ergonomics Society. 1997 Jun;39(2):230-53. Available from: https://journals.sagepub.com/doi/10.1518/001872097778543886.
[27] Hancock PA, Billings DR, Schaefer KE, Chen JYC, de Visser EJ, Parasuraman R. A Meta-Analysis of Factors Affecting Trust in Human-Robot Interaction. Human Factors. 2011;53(5):517-27. PMID: 22046724. Available from: https://doi.org/10.1177/0018720811417254.
[28] Khavas ZR, Ahmadzadeh SR, Robinette P. Modeling Trust in Human-Robot Interaction: A Survey. In: Wagner AR, Feil-Seifer D, Haring KS, Rossi S, Williams T, He H, et al., editors. Social Robotics. Lecture Notes in Computer Science. Cham: Springer International Publishing; 2020. p. 529-41.
[29] Yagoda RE. Development of the Human Robot Interaction Workload Measurement Tool (HRI-WM). Proceedings of the Human Factors and Ergonomics Society Annual Meeting. 2010;54(4):304-8. Available from: https://doi.org/10.1177/154193121005400408.
[30] Yagoda RE, Gillan DJ. You Want Me to Trust a ROBOT? The Development of a Human–Robot Interaction Trust Scale. International Journal of Social Robotics. 2012 Aug;4(3):235-48. Available from: https://doi.org/10.1007/s12369-012-0144-0.
[31] Schaefer KE. Measuring Trust in Human Robot Interactions: Development of the "Trust Perception Scale-HRI". In: Mittu R, Sofge D, Wagner A, Lawless WF, editors. Robust Intelligence and Trust in Autonomous Systems. Boston, MA: Springer US; 2016. p. 191-218. Available from: http://link.springer.com/10.1007/978-1-4899-7668-0_10.
[32] Guo Y, Yang XJ. Modeling and Predicting Trust Dynamics in Human–Robot Teaming: A Bayesian Inference Approach. International Journal of Social Robotics. 2020. Publisher: Springer Science and Business Media B.V.
[33] Wang Q, Liu D, Carmichael MG, Aldini S, Lin CT. Computational Model of Robot Trust in Human Co-Worker for Physical Human-Robot Collaboration. IEEE Robotics and Automation Letters. 2022;7(2):3146-53.
[34] Mizanoor Rahman SM, Wang Y, Walker ID, Mears L, Pak R, Remy S. Trust-based compliant robot-human handovers of payloads in collaborative assembly in flexible manufacturing. In: 2016 IEEE International Conference on Automation Science and Engineering (CASE); 2016. p. 355-60.
[35] Xu A, Dudek G. OPTIMo: Online Probabilistic Trust Inference Model for Asymmetric Human-Robot Collaborations. In: Proceedings of the Tenth Annual ACM/IEEE International Conference on Human-Robot Interaction. Portland Oregon USA: ACM; 2015. p. 221-8. Available from: https://dl.acm.org/doi/10.1145/2696454.2696492.
[36] Bellman R. A Markovian decision process. Journal of Mathematics and Mechanics. 1957;6(5):679-84. Available from: http://www.jstor.org/stable/24900506.
[37] Nam C, Walker P, Li H, Lewis M, Sycara K. Models of Trust in Human Control of Swarms With Varied Levels of Autonomy. IEEE Transactions on Human-Machine Systems. 2020 Jun;50(3):194-204. Conference Name: IEEE Transactions on Human-Machine Systems.
[38] Fung HL, Darvariu VA, Hailes S, Musolesi M. Trust-based Consensus in Multi-Agent Reinforcement Learning Systems. arXiv; 2022. ArXiv:2205.12880 [cs]. Available from: http://arxiv.org/abs/2205.12880.
[39] Frattolillo F, Brandizzi N, Cipollone R, Iocchi L. Towards Computational Models for Reinforcement Learning in Human-AI Teams. In: Proceedings of the 2nd International Workshop on Multidisciplinary Perspectives on Human-AI Team Trust. vol. 3634. Gothenburg, Sweden: CEUR Workshop Proceedings; 2023. p. 41-51. Available from: https://ceur-ws.org/Vol-3634/.
[40] Di Rienzo S, Frattolillo F, Cipollone R, Fanti A, Brandizzi N, Iocchi L. Developing Targeted Communication through a Trust Factor in Multi-Agent Reinforcement Learning. In: International Conference on Hybrid Human-Artificial Intelligence 2024; 2024. .

[41] Frey V, Martinez J. Interpersonal trust modelling through multi-agent Reinforcement Learning. Cognitive Systems Research. 2024 Jan;83:101157. Available from: https://www.sciencedirect.com/science/article/pii/S1389041723000918.
[42] Littman ML. Markov games as a framework for multi-agent reinforcement learning. Mach Learn Proc 1994. 1994:157-63.
[43] Fikes RE, Nilsson NJ. Strips: A new approach to the application of theorem proving to problem solving. Artificial Intelligence. 1971;2(3):189-208. Available from: https://www.sciencedirect.com/science/article/pii/0004370271900105.
[44] Kuběna AA. Pexeso ("Concentration game") as an arbiter of bounded-rationality models. In: Proceedings of the 28th International Conference on Mathematical Methods in; 2010. .
[45] Foerster KT, Wattenhofer R. The solitaire memory game. ETH Zurich; 2013.
[46] Rillig MC, Ågerstrand M, Bi M, Gould KA, Sauerland U. Risks and benefits of large language models for the environment. Environmental Science & Technology. 2023;57(9):3464-6.
[47] Touvron H, Lavril T, Izacard G, Martinet X, Lachaux MA, Lacroix T, et al.. LLaMA: Open and Efficient Foundation Language Models; 2023. Available from: https://arxiv.org/abs/2302.13971.
[48] Jiang AQ, Sablayrolles A, Mensch A, Bamford C, Chaplot DS, de las Casas D, et al.. Mistral 7B; 2023. Available from: https://arxiv.org/abs/2310.06825.
[49] Bai J, Bai S, Chu Y, Cui Z, Dang K, Deng X, et al.. Qwen Technical Report; 2023. Available from: https://arxiv.org/abs/2309.16609.
[50] Watkins CJCH. Learning from delayed rewards. King's College, Cambridge United Kingdom; 1989.
[51] Watkins CJ, Dayan P. Q-learning. Machine learning. 1992;8:279-92.
[52] Haslum P, Lipovetzky N, Magazzeni D, Muise C. An Introduction to the Planning Domain Definition Language. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers; 2019. Available from: https://doi.org/10.2200/S00900ED2V01Y201902AIM042.
[53] OpenAI. Chatgpt: Optimizing language models for dialogue.; 2022. Accessed on 2025-01-29. https://openai.com/index/chatgpt/.